DLDCA Outlab: Wires, Regs, and Time

Week 2

August 17, 2026

Introduction

In this outlab we look closely at two distinctions that are easy to mix up when you are new to Verilog: `wire` vs `reg`, and combinational vs sequential logic. These are *not* the same distinction — `wire`/`reg` is a rule about how a signal is allowed to be driven in the source code, while combinational/sequential is a property of the hardware that gets synthesized. The six tasks in this lab build on each other: we start by writing the same tiny circuit two different ways, then add real memory, then deliberately break things so you can recognize the failure modes, and finish by designing a small digital stopwatch from scratch.

Necessary tools

We shall be using `icarus-verilog` (or) `iverilog` in order to compile verilog HDL (Hardware Description Language). To visualize waveforms we shall use the VS Code extension 'Vaporview'. Before starting the lab, ensure that both tools are installed in your system. This can be verified by running the following:

```
$ iverilog
iverilog: no source files.

Usage: iverilog [-EiRSuvV] [-B base] [-c cmdfile|-f cmdfile]
                [-g1995|-g2001|-g2005|-g2005-sv|-g2009|-g2012] [-g<feature>]
                [-D macro[=defn]] [-I includedir] [-L moduledir]
                [-M [mode=]depfile] [-m module]
                [-N file] [-o filename] [-p flag=value]
                [-s topmodule] [-t target] [-T min|typ|max]
                [-W class] [-y dir] [-Y suf] [-l file] source_file(s)

See the man page for details.
```

Structure of submission/templates

Submission format:

```
your-roll-no/
|- task1/
    |- mux_dataflow.v (TODO)
    |- tb_task1.v
|- task2/
    |- mux_dataflow.v
    |- mux_behavioral.v (TODO)
    |- tb_task2.v
|- task3/
    |- mux_behavioral.v
    |- loadable_reg.v (TODO)
    |- tb_task3.v
|- task4/
    |- buggy_regs.v
    |- fixed_regs.v (TODO)
    |- answers.txt (TODO)
    |- tb_task4.v
|- task5/
    |- sat_counter.v (TODO)
    |- tb_task5.v
|- task6/
    |- tick_gen.v
    |- stopwatch.v (TODO)
    |- tb_task6.v
```

The folder structure of the expected submission for this outlab is given above. We expect this folder to be compressed into a tarball with the naming convention of `your-roll-no.tar.gz`. The command given below can be used to do the same

```
$ tar -cvzf your-roll-no.tar.gz your-roll-no/
```

Tasks

Task 1: A Simple Mux, Dataflow Style

The goal of this task is to create a 2:1 multiplexer. The module has two data inputs `a` and `b`, a select line `sel`, and a single output `y`. When `sel` is 0, `y` should follow `a`; when `sel` is 1, `y` should follow `b`.

sel	a	b	y
0	0	x	0
0	1	x	1
1	x	0	0
1	x	1	1

You have been provided a template module file (`mux_dataflow.v`). Implement the mux using a single `assign` statement — no `always` block, and every signal in this file should be a `wire`. A testbench (`tb_task1.v`) has been provided to check your implementation.

To run your code, use the makefile:

```
$ make task1
```

This will open a waveform viewer within VSC where you can see the output.

Keep this module around — you will reuse your completed `mux_dataflow.v` in Task 2.

Task 2: The Same Mux, Behavioral Style

You just built a 2:1 mux using `assign` and `wire`. Now build the *exact same circuit* again, but this time using an `always @(*)` block, with `y` declared as a `reg`.

Your completed `mux_dataflow.v` from Task 1 has been copied into this folder. You should not need to modify it — fill in the template `mux_behavioral.v` instead, using an `if/else` (or equivalent) inside an `always @(*)` block to drive `y`.

The provided testbench (`tb_task2.v`) instantiates *both* modules side by side, drives them with identical inputs, and checks that their outputs agree at every step, printing `PASS` or `FAIL` at the end.

Before you run the simulation, answer this for yourself: do you expect these two modules to synthesize to the same hardware, different hardware, or is one of them not really combinational at all? Write a one or two sentence answer as a comment at the top of `mux_behavioral.v`.

To run your code, use the makefile:

```
$ make task2
```

This will open a waveform viewer within VSC where you can see the output, and the terminal will print whether the two implementations matched.

Task 3: A Loadable Register

Every `reg` you have written so far (in this outlab) has secretly been combinational — a `reg` does not, by itself, mean "this becomes a flip-flop." In this task you will write your first `reg` that is an actual piece of memory.

Build a 1-bit loadable register: on every rising edge of `clk`, if `rst` is high the output `q` should synchronously reset to 0; otherwise, if `load` is high, `q` should take the value of `d`; otherwise, `q` should hold its current value.

Your completed `mux_behavioral.v` from Task 2 has been copied into this folder. Use it to compute the *next value* of `q` (selecting between "hold `q`" and "load `d`" using `load` as the select line), and only register that value inside an `always @(posedge clk)` block. In other words: the decision of *what* the next value should be is combinational logic you already have; the only new thing in this task is *when* that value actually takes effect.

A testbench (`tb_task3.v`) is provided that walks through holding, loading, and resetting the register.

Before moving on, make sure you can answer: what changed between this `reg` and every earlier `reg` you've written in this lab? Why does *this* one actually need a clock edge to make sense?

To run your code, use the makefile:

```
$ make task3
```

Task 4: Diagnosis

`buggy_regs.v` (provided, do not edit) contains four variants of Task 3's loadable register, `reg_bug1` through `reg_bug4`. Each one is structured the same way — a combinational block computes `next_q`, and a sequential block registers `next_q` into `q` on `posedge clk` — and each one has exactly one bug.

For each module, fill in `answers.txt` with:

- whether the block that's supposed to be combinational actually is, and why or why not
- precisely what is wrong with the code
- what you would actually observe in simulation or in synthesized hardware as a result

Then write corrected versions of all four modules in `fixed_regs.v`, named `reg_bug1_fixed` through `reg_bug4_fixed`. Each corrected module should behave identically to Task 3's `loadable_reg`: synchronous reset takes priority over `load`, and `q` holds its value when `load` is low.

A testbench (`tb_task4.v`) is provided that drives all four fixed modules in parallel, including a case where `rst` and `load` are asserted on the same clock edge.

A note on one of the four bugs: not every bug in this file will necessarily cause a visible mismatch against the provided testbench when the module is tested on its own. Passing the testbench is necessary but not sufficient — your written explanation in `answers.txt` is what actually demonstrates you understand each bug, so don't just rely on the simulation result.

To run your code, use the makefile:

```
$ make task4
```

Task 5: Saturating Up/Down Counter

So far you have filled in one half of the comb-next-state/seq-register pattern at a time — the combinational piece, or the sequential piece, was mostly given to you. In this task you design both halves yourself.

Build a 4-bit saturating up/down counter, `sat_counter`, with the following behavior on every rising edge of `clk`:

- if `rst` is high, `count` synchronously resets to 0, overriding everything else
- if `up` is high and `down` is low, `count` increments — unless it is already at 15, in which case it *stays* at 15 (it does not wrap around to 0)
- if `down` is high and `up` is low, `count` decrements — unless it is already at 0, in which case it *stays* at 0
- if `up` and `down` are equal (both 0 or both 1), `count` holds its current value

Structure your implementation the same way as Task 3 and Task 4: a combinational block (driving a `wire`) that computes what the next value of `count` *should* be, and a separate `always @(posedge clk)` block that does nothing but register that value. Nothing that decides "what

should happen" belongs inside the clocked block, and nothing that "remembers" anything belongs in the combinational block.

A testbench (`tb_task5.v`) is provided that checks counting, both saturation edges, holding, and reset.

To run your code, use the makefile:

```
$ make task5
```

Task 6: A BCD Digital Stopwatch

Time to put everything together. You will build a simple digital stopwatch that displays minutes and seconds as four BCD (binary-coded decimal) digits: `min_tens`, `min_ones`, `sec_tens`, `sec_ones`, each a 4-bit value from 0 to 9.

A completed tick generator, `tick_gen.v`, has been provided. In real hardware you would set its `LIMIT` parameter so that `tick` pulses once per second (e.g. `LIMIT` = your clock frequency in Hz); for simulation, instantiate it with a small `LIMIT` (we suggest 4) so you don't have to wait for millions of clock edges to see the stopwatch move. You do not need to modify `tick_gen.v`. `stopwatch.v` has an extra output port, `tick_out`, purely so the testbench can directly observe when a tick occurs — wire your instantiated `tick_gen`'s `tick` output to it. A real stopwatch chip wouldn't need this pin; it exists here only for testability.

Build `stopwatch.v` with the following behavior:

- `start_stop` is a one-clock-cycle pulse representing a button press — each press should toggle whether the stopwatch is running. You will need exactly one `reg` to remember this.
- While running, on every `tick`, the seconds-ones digit increments. When it would go past 9, it rolls to 0 and carries into the seconds-tens digit.
- The seconds-tens digit rolls over at 6 (since seconds only run 0–59), carrying into the minutes-ones digit, which behaves the same way as seconds-ones. The minutes-tens digit rolls over at 6 as well, wrapping the whole display back to `00:00`.
- `rst` synchronously zeroes every digit, regardless of whether the stopwatch is running.

As with Task 5, structure this as a combinational block that decides, for each digit, whether it should roll over and whether it should carry into the next digit, and a separate sequential block that does nothing but apply those decisions on the clock edge. Nothing that "decides" belongs in the clocked block.

A testbench (`tb_task6.v`) is provided, including a check of the full `59:59` → `00:00` rollover.

Written justification (required): in a comment at the top of `stopwatch.v`, answer in 2–3 sentences: why is `running` a `reg`? Why is the carry signal out of `sec_ones` naturally a `wire`, even though it's computed from values that are themselves `reg`s?

To run your code, use the makefile:

```
$ make task6
```